LadderLens

A Benchmark and Decision Toolkit forPQC DNSSEC Merkle Tree Ladder (MTL) Signatures

PROBLEM STATEMENT:

PS-036, PQC DNSSEC MTL Benchmark Using a Reference Implementation

TEAM NAME: Nexora

TEAM MEMBERS: Dr Mukesh Kumar
Aditya Goyal
Akshat Dubey

REPOSITORY: https://github.com/aiori-hackathon/A3- PS036-TC187

EVENT: AIORI Hackathon

Contents

01. Introduction: 01
- Project Overview & Objectives
- Project Scope and Team Details

02. Table of Contents: 02
- Document Structure

03. Problem Description: 03
- Quantum Threat to Classical DNSSEC
- PQC Signature Size and UDP Limitations
- Project Scope and Limitations

04. Solution Proposed: 04
- Verisign MTL Reference Implementation
- C Benchmark Harness and Python Runner

05. Optimizations Proposed by the Team: 05
- Statistical Analysis and Confidence Intervals
- Operation-Count Cost Mode

06. Solution Architecture and Design: 06
- Pinned Environment and Reference Implementation
- Signing, Verification and Byte-Count Harness
- Python Runner and Analysis Pipeline
- Report, Dashboard and Integrity Validation

07. Timeline of Delivery: 07
- Development and CI Setup
- Benchmarking and Statistical Analysis
- Cache Evaluation and Dashboard Development
- Final Testing and Hackathon Submission

08. References: 08
- IETF MTL Draft and Hackathon Materials
- Verisign MTL Reference Implementation
- NIST FIPS 205 and DNS/EDNS Standards

09. Conclusion : 09
- Project Summary and Significance
- Key Findings and Performance Evaluation

Problem Description

- Classical public-key algorithms used in DNSSEC are vulnerable to future quantum cryptanalysis.
- Post-quantum (PQC) signatures resist quantum attacks, but their signatures and keys are much larger.
- Large signatures can push DNS UDP responses past common size limits (512 B, 1,232 B, 1,472 B), causing fragmentation, truncation with TCP fallback, or dropped lookups.
- The IETF MTL mode (draft-harvey-cfrg-mtl-mode) signs records in batches using a Merkle tree ladder with compact authentication paths. Its benefit depends on the ladder being cached and reused.
- The project asks how signing time, verification time, and transmitted bytes compare with and without MTL across three dataset sizes in a repeatable pinned environment, and how measured cache usage differs from analytical estimates.

MEASURED EXAMPLE:

- **REFERENCE IMPLEMENTATION**
- **N = 1**

Onefullsignature:
76 B condensed signature
+ 7,928 B ladder.

8,004 B is about 6.5× the 1,232 B UDP limit

Scope limits from the problem statement

- No new cryptography.
- No resolver written from scratch. No Internet deployment.
- No claimed security proof.

Solution Proposed

- **Reference implementation**

  Verisign MTL v1.3.1 is used with the SLH-DSA-SHA2-128s-MTL-SHA2-128 algorithm. The benchmark follows the problem statement's scope: one post-quantum signature algorithm and one reference implementation. The cryptographic implementation itself remains unchanged, so results reflect the reference implementation rather than a newly designed scheme.

- **C benchmark harness**

  The C harness measures signing time, verification time, and transmitted signature-related bytes for MTL and the documented baseline configuration. It provides the low-level timing and size measurements and records results in a structured format for later analysis. It is the only component that directly interacts with the reference implementation.

- **Python runner**

  The Python runner automates repeated trials across the proposed dataset sizes N = 1, 10, and 1,000 (to be confirmed with the mentor). It schedules runs, records configuration details and a CPU fingerprint, stores raw measurements in CSV files, and generates summary tables. Keeping execution automated makes experiments easier to repeat and compare.

- **Cache model**

  Cache behaviour is evaluated using two clearly separated views: directly measured cache behaviour and an analytical estimate. The model explores how ladder reuse and cache hit rate affect amortized cost. Estimated values are labelled as estimates and are never presented as measured results.

- **Pinned environment**

  A Docker image is built and tested through GitHub Actions CI. Final benchmark numbers are collected in one fixed environment, such as GitHub Codespaces, with the environment and CPU fingerprint recorded. CI timings are used as sanity checks only; they are not mixed with the final performance measurements.

- **Reproducibility outputs**

  The project generates an automated report, a summary table, a dashboard, demo.sh, and a README. A SHA-256 reproducibility manifest records the binaries, inputs, configurations, and results. The make verify command checks the manifest so another person can verify that the reported outputs correspond to the recorded experiment artifacts.

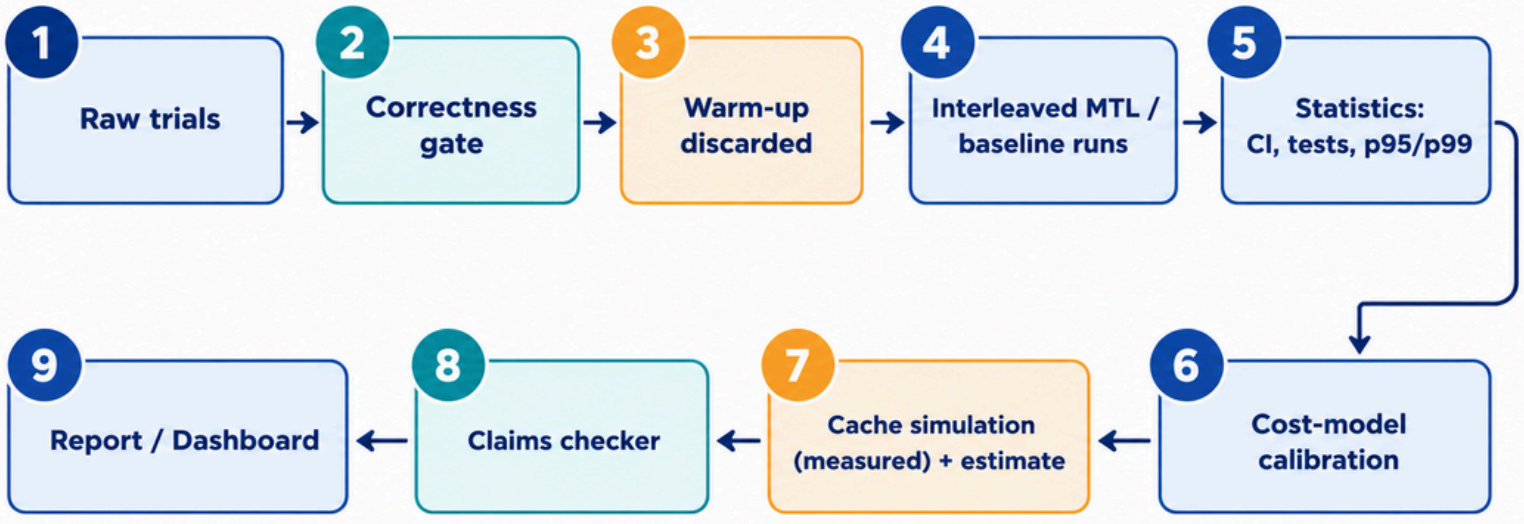
- **Baseline clarification**

  The current non-MTL comparison uses MTL at N = 1 (approximately 8,004 bytes versus 7,856 bytes), not plain SLH-DSA. This limitation is stated explicitly in the report. Any per-record comparison against plain SLH-DSA will only be claimed if that baseline is implemented and measured esparately.

Optimizations Proposed by NEXORA:

#	Optimization	Before (baseline benchmark)	After (planned)
1	Statistical rigor	Single timings, noisy	Interleaved trials, warm-up discarded, bootstrap CI, Mann–Whitney, p95/p99, noise floor
2	Operation-count cost model	Time depends on machine	Hash-invocation counts vs N, calibrated against measured time
3	Cache evaluation	Analytical assumption only	LRU simulator on Zipf/uniform traces; measured hit rate beside analytical estimate
4	Batch tradeoff analysis	Only size and time shown	Frontier of N vs cost vs bytes vs staleness window
5	Reproducibility ledger	Manual notes	SHA-256 manifest of binaries, inputs, configs, results; rechecked by make verify
6	Claims-to-evidence checker	Honesty by discipline	Build fails if any number is untagged measured/estimated or lacks a source reference
7	Correctness gates	Timing only	Test vectors and tampered-input negative tests run before timing
8	Wire-size and UDP survival	Raw signature bytes	Exact message sizes via dnspython (header, EDNS/DO) against 512/1232/1472 B
9	Break-even explorer	Single-point comparison	N × cache-hit-rate map showing where MTL pays off
10	Ladder visualizer	Static description	Interactive view of transmitted ladder and path nodes with byte costs

Optimization Process Flow:



Analytical items are labeled as estimates in the report.

Solution Architecture and Design

01 Pinned environment
Docker + CI, fixed host, CPU fingerprint

↓

02 Reference implementation
Verisign MTL v1.3.1 · SLH-DSA-SHA2-128s-MTL-SHA2-128

↓

03 C harness
Sign / verify / byte counts for MTL and baseline

↓

04 Python runner
Three dataset sizes × repeated trials → CSV

↓

05 Analysis layer
Statistics · cost model · cache simulator · UDP/wire-size analysis

↓

06 Outputs
Summary table · auto-generated report · dashboard · demo.sh

↓

07 Integrity checks
Reproducibility manifest · make verify · claims checker

COMPONENT RESPONSIBILITIES

Harness (C)
Only component touching the reference implementation; does not modify cryptography.

Runner (Python)
Schedules trials and records configuration plus CPU fingerprint.

Analysis layer
Separates measured outputs from analytical outputs at the data level.

Presentation layer
Report and dashboard draw numbers only from tagged result files.

Analysis layer
Separates measured outputs from analytical outputs at the data level.

Report layer
Report and dashboard draw numbers only from tagged result files.

Timeline of Delivery

Date & Day	Task / Phase	Details
10 Oct Day 1	**Repository Ownership & Project Initiation** ✓ Confirmed	• Transfer ownership of the AIORI repository. • Establish repository access and responsibility for project development. • Begin aligning the repository with PS-036 requirements. **Milestone: Repository ownership transferred.**
11–12 Oct Day 2–3	**Requirements, Scope & Architecture** Initial planning phase	• Define the three benchmark arms: plain SLH-DSA, MTL at N=1, and batched MTL. • Specify signing time, verification time, signature bytes, and cold/warm cache measurements. • Plan the C harness, Python runner, CSV output, analysis, and report workflow. **Deliverable: Agreed benchmark design and task breakdown.**
13–14 Oct Day 4–5	**Reference Implementation & C Harness**	• Build and test the Verisign MTL v1.3.1 reference environment. • Develop the measurement harness for signing, verification, and byte counts. • Reproduce and validate the N=1 baseline. **Deliverable: Working harness and initial benchmark results.**
15–16 Oct Day 6–7	**Measurement Logging & Verification**	• Define a consistent CSV schema for each trial, arm, and dataset size. • Record signing time, verification time, ladder size, and authentication-path bytes. • Implement separate cold-cache and warm-cache verification runs. **Deliverable: Structured, machine-readable benchmark data.**
17–18 Oct Day 8–9	**Python Runner & Multi-Size Benchmarking**	• Automate repeated trials using the Python runner. • Configure dataset sizes N = 10, 100, and 1,000, subject to mentor confirmation. • Interleave benchmark arms to reduce ordering bias from machine load. **Deliverable: Automated benchmark workflow across dataset sizes.**
19–20 Oct Day 10–11	**Controlled Benchmark Runs & Reproducibility**	• Run benchmarks in one fixed environment. • Capture raw CSV results and a reproducibility manifest with CPU and software details. • Validate output consistency and rerun failed or incomplete trials. **Deliverable: Validated raw results and environment manifest.**
21–22 Oct Day 12–13	**Performance Analysis & UDP-Fit Evaluation**	• Calculate medians and ranges for measured timing and size results. • Compare per-record signatures with cold and warm MTL answers. • Evaluate packet-size fit against the 512 B, 1,232 B, and 1,472 B reference limits. **Deliverable: Analysis tables and initial comparison charts.**
23–24 Oct Day 14–15	**Cache Model & Break-Even Analysis**	• Calculate how many answers must reuse a cached ladder before MTL saves cumulative bytes. • Account for the full ladder cost, not just the short authentication path. • Clearly label model-derived results as analytical, not measured. **Deliverable: Break-even chart and cache-reuse analysis.**
25–26 Oct Day 16–17	**Report Generation & Documentation**	• Generate the report PDF with tables, charts, methodology, and limitations. • Document installation, execution, benchmark configuration, and interpretation. • Prepare the README and example outputs. **Deliverable: Draft technical report and documentation.**
27–28 Oct Day 18–19	**Integration, CI & End-to-End Validation**	• Test the full workflow using ./demo.sh. • Validate Docker setup, CI execution, and make verify. • Capture the CSV excerpt, break-even chart, and other demo evidence. **Deliverable: Submission-ready repository and reproducible demo.**
29 Oct Day 20	**Final Review & Submission Audit**	• Check all figures, source references, labels, and known limitations. • Confirm team details, repository URL, project code, and required files. • Conduct a final demonstration rehearsal. **Deliverable: Final reviewed submission package.**
30 Oct Day 21	**Final Submission & Demonstration**	• Submit the final project within the official deadline. • Demonstrate the benchmark workflow and explain the measured findings. • Keep time available for last-minute fixes and submission checks. **Milestone: Final PS-036 submission.**

References

01	IETF Draft	PQC DNSSEC MTL Mode (draft-harvey-cfrgmtl-mode).
02	IETF hackathon	IETF 126 Hackathon materials.
03	Reference implementation	Verisign MTL reference implementation, v1.3.1.
04	PGC standard	NIST FIPS 205 (SLH-DSA).
05	DNS / EDNS standards	Relevant standards for UDP size limits (for example, RFC 6891). Verify the exact RFC list before final submission.
06	Tools	Docker, GitHub Actions, GitHub Codespaces, Python, pandas, dnspython, and Streamlit.
07	Problem statement	PS-036, AIORI-3.
08	Project repository	https://github.com/aiori-hackathon/A3-PS036-TC187

Conclusion

LADDERLENS OVERVIEW

1. PQC Benchmarking: Compares standard SLH-DSA signatures with Merkle Tree Ladder (MTL) approaches for DNSSEC.
2. Performance Measurement: Measures signing time, verification time, transmitted bytes, and cache usage.
3. Network Efficiency: Analyzes UDP packet-size limits and the impact of cache reuse on data transmission.
4. Automated Reporting: Generates benchmark results, comparison charts, and cache-reuse break-even analysis for informed decision-making.

Significance

The toolkit gives DNS operators and standards participants an evidence-based, reproducible view of when MTL helps and when it does not, without claiming security results.

Known limits

- Zone churn is unsupported in the reference implementation.
- The non-MTL baseline is MTL at N = 1 unless a plain per-record baseline is added.
- The earlier 25–80% figure is an estimate until the batch sweep measures it.
- CI timing are sanity checks only.

Next steps

Complete the official repo and CI, run final measurements in the fixed environment, generate the report, and present the demo with a pseudocode snippet to the mentor.